AuthXConfig

class authx.config.AuthXConfig

```
AuthXConfig(__pydantic_self__, _case_sensitive=None, _nested_model_default_partial_update=None,
_env_prefix=None, _env_file=ENV_FILE_SENTINEL, _env_file_encoding=None, _env_ignore_empty=None,
_env_nested_delimiter=None, _env_nested_max_split=None, _env_parse_none_str=None,
_env_parse_enums=None, _cli_prog_name=None, _cli_parse_args=None, _cli_settings_source=None,
_cli_parse_none_str=None, _cli_hide_none_type=None, _cli_avoid_json=None,
_cli_enforce_required=None, _cli_use_class_docs_for_groups=None, _cli_exit_on_error=None,
_cli_prefix=None, _cli_flag_prefix_char=None, _cli_implicit_flags=None,
_cli_ignore_unknown_args=None, _cli_kebab_case=None, _cli_shortcuts=None, _secrets_dir=None,
**values)
```

Bases: `BaseSettings`

AuthX Base Configuration Object.

PARAMETER	DESCRIPTION
`BaseSettings`	BaseSettings class from Pydantic **TYPE:** `BaseSettings`

RAISES	DESCRIPTION
`BadConfigurationError`	If JWT_ALGORITHM is not supported
`BadConfigurationError`	If JWT_ALGORITHM requires a key and it is not set

RETURNS	DESCRIPTION
`AuthXConfig`	AuthX Configuration Object

Source code in `.venv/lib/python3.14/site-packages/pydantic_settings/main.py`

```python
def __init__(
    __pydantic_self__,
    _case_sensitive: bool | None = None,
    _nested_model_default_partial_update: bool | None = None,
    _env_prefix: str | None = None,
    _env_file: DotenvType | None = ENV_FILE_SENTINEL,
    _env_file_encoding: str | None = None,
    _env_ignore_empty: bool | None = None,
    _env_nested_delimiter: str | None = None,
    _env_nested_max_split: int | None = None,
    _env_parse_none_str: str | None = None,
    _env_parse_enums: bool | None = None,
    _cli_prog_name: str | None = None,
    _cli_parse_args: bool | list[str] | tuple[str, ...] | None = None,
    _cli_settings_source: CliSettingsSource[Any] | None = None,
    _cli_parse_none_str: str | None = None,
    _cli_hide_none_type: bool | None = None,
    _cli_avoid_json: bool | None = None,
    _cli_enforce_required: bool | None = None,
    _cli_use_class_docs_for_groups: bool | None = None,
    _cli_exit_on_error: bool | None = None,
    _cli_prefix: str | None = None,
    _cli_flag_prefix_char: str | None = None,
    _cli_implicit_flags: bool | None = None,
    _cli_ignore_unknown_args: bool | None = None,
    _cli_kebab_case: bool | None = None,
    _cli_shortcuts: Mapping[str, str | list[str]] | None = None,
    _secrets_dir: PathType | None = None,
    **values: Any,
) -> None:
    super().__init__(
        **__pydantic_self__._settings_build_values(
            values,
            _case_sensitive=_case_sensitive,
            _nested_model_default_partial_update=_nested_model_default_partial_update,
            _env_prefix=_env_prefix,
            _env_file=_env_file,
            _env_file_encoding=_env_file_encoding,
            _env_ignore_empty=_env_ignore_empty,
            _env_nested_delimiter=_env_nested_delimiter,
            _env_nested_max_split=_env_nested_max_split,
            _env_parse_none_str=_env_parse_none_str,
            _env_parse_enums=_env_parse_enums,
            _cli_prog_name=_cli_prog_name,
            _cli_parse_args=_cli_parse_args,
            _cli_settings_source=_cli_settings_source,
            _cli_parse_none_str=_cli_parse_none_str,
            _cli_hide_none_type=_cli_hide_none_type,
            _cli_avoid_json=_cli_avoid_json,
            _cli_enforce_required=_cli_enforce_required,
            _cli_use_class_docs_for_groups=_cli_use_class_docs_for_groups,
            _cli_exit_on_error=_cli_exit_on_error,
            _cli_prefix=_cli_prefix,
            _cli_flag_prefix_char=_cli_flag_prefix_char,
            _cli_implicit_flags=_cli_implicit_flags,
            _cli_ignore_unknown_args=_cli_ignore_unknown_args,
            _cli_kebab_case=_cli_kebab_case,
            _cli_shortcuts=_cli_shortcuts,
            _secrets_dir=_secrets_dir,
        )
    )
```

attr JWT_ACCESS_TOKEN_EXPIRES `class-attribute` `instance-attribute`

```python
JWT_ACCESS_TOKEN_EXPIRES = timedelta(minutes=15)
```

attr JWT_ALGORITHM `class-attribute` `instance-attribute`

```
JWT_ALGORITHM = 'HS256'
```

attr **JWT_DECODE_ALGORITHMS** `class-attribute` `instance-attribute`

```
JWT_DECODE_ALGORITHMS = Field(default_factory=lambda: ['HS256'])
```

attr **JWT_DECODE_AUDIENCE** `class-attribute` `instance-attribute`

```
JWT_DECODE_AUDIENCE = None
```

attr **JWT_DECODE_ISSUER** `class-attribute` `instance-attribute`

```
JWT_DECODE_ISSUER = None
```

attr **JWT_DECODE_LEEWAY** `class-attribute` `instance-attribute`

```
JWT_DECODE_LEEWAY = 0
```

attr **JWT_ENCODE_AUDIENCE** `class-attribute` `instance-attribute`

```
JWT_ENCODE_AUDIENCE = None
```

attr **JWT_ENCODE_ISSUER** `class-attribute` `instance-attribute`

```
JWT_ENCODE_ISSUER = None
```

attr **JWT_ENCODE_NBF** `class-attribute` `instance-attribute`

```
JWT_ENCODE_NBF = True
```

attr **JWT_ERROR_MESSAGE_KEY** `class-attribute` `instance-attribute`

```
JWT_ERROR_MESSAGE_KEY = 'msg'
```

attr **JWT_IDENTITY_CLAIM** `class-attribute` `instance-attribute`

```
JWT_IDENTITY_CLAIM = 'sub'
```

attr **JWT_PRIVATE_KEY** `class-attribute` `instance-attribute`

```
JWT_PRIVATE_KEY = None
```

attr **JWT_PUBLIC_KEY** `class-attribute` `instance-attribute`

```
JWT_PUBLIC_KEY = None
```

attr **JWT_PREVIOUS_SECRET_KEY** `class-attribute` `instance-attribute`

```
JWT_PREVIOUS_SECRET_KEY = None
```

attr **JWT_PREVIOUS_PUBLIC_KEY** `class-attribute` `instance-attribute`

```
JWT_PREVIOUS_PUBLIC_KEY = None
```

attr **JWT_REFRESH_TOKEN_EXPIRES** `class-attribute` `instance-attribute`

```
JWT_REFRESH_TOKEN_EXPIRES = timedelta(days=20)
```

attr **JWT_SECRET_KEY** `class-attribute` `instance-attribute`

```
JWT_SECRET_KEY = None
```

attr **JWT_TOKEN_LOCATION** `class-attribute` `instance-attribute`

```
JWT_TOKEN_LOCATION = Field(default_factory=lambda: ['headers'])
```

attr **JWT_HEADER_NAME** `class-attribute` `instance-attribute`

```
JWT_HEADER_NAME = 'Authorization'
```

attr **JWT_HEADER_TYPE** `class-attribute` `instance-attribute`

```
JWT_HEADER_TYPE = 'Bearer'
```

attr **JWT_ACCESS_COOKIE_NAME** `class-attribute` `instance-attribute`

```
JWT_ACCESS_COOKIE_NAME = 'access_token_cookie'
```

attr **JWT_ACCESS_COOKIE_PATH** `class-attribute` `instance-attribute`

```
JWT_ACCESS_COOKIE_PATH = '/'
```

attr **JWT_COOKIE_CSRF_PROTECT** `class-attribute` `instance-attribute`

```
JWT_COOKIE_CSRF_PROTECT = True
```

attr **JWT_COOKIE_DOMAIN** `class-attribute` `instance-attribute`

```
JWT_COOKIE_DOMAIN = None
```

attr **JWT_COOKIE_MAX_AGE** `class-attribute` `instance-attribute`

```
JWT_COOKIE_MAX_AGE = None
```

attr **JWT_COOKIE_SAMESITE** `class-attribute` `instance-attribute`

```
JWT_COOKIE_SAMESITE = 'lax'
```

attr **JWT_COOKIE_SECURE** `class-attribute` `instance-attribute`

```
JWT_COOKIE_SECURE = True
```

attr **JWT_COOKIE_HTTP_ONLY** `class-attribute` `instance-attribute`

```
JWT_COOKIE_HTTP_ONLY = True
```

attr **JWT_REFRESH_COOKIE_NAME** `class-attribute` `instance-attribute`

```
JWT_REFRESH_COOKIE_NAME = 'refresh_token_cookie'
```

attr **JWT_REFRESH_COOKIE_PATH** `class-attribute` `instance-attribute`

```
JWT_REFRESH_COOKIE_PATH = '/'
```

attr **JWT_SESSION_COOKIE** `class-attribute` `instance-attribute`

```
JWT_SESSION_COOKIE = True
```

attr **JWT_ACCESS_CSRF_COOKIE_NAME** `class-attribute` `instance-attribute`

```
JWT_ACCESS_CSRF_COOKIE_NAME = 'csrf_access_token'
```

attr **JWT_ACCESS_CSRF_COOKIE_PATH** `class-attribute` `instance-attribute`

```
JWT_ACCESS_CSRF_COOKIE_PATH = '/'
```

attr **JWT_ACCESS_CSRF_FIELD_NAME** `class-attribute` `instance-attribute`

```
JWT_ACCESS_CSRF_FIELD_NAME = 'csrf_token'
```

attr **JWT_ACCESS_CSRF_HEADER_NAME** `class-attribute` `instance-attribute`

```
JWT_ACCESS_CSRF_HEADER_NAME = 'X-CSRF-TOKEN'
```

attr **JWT_CSRF_CHECK_FORM** `class-attribute` `instance-attribute`

```
JWT_CSRF_CHECK_FORM = False
```

attr **JWT_CSRF_IN_COOKIES** `class-attribute` `instance-attribute`

```
JWT_CSRF_IN_COOKIES = True
```

attr **JWT_CSRF_METHODS** `class-attribute` `instance-attribute`

```
JWT_CSRF_METHODS = Field(default_factory=lambda: ['POST', 'PUT', 'PATCH', 'DELETE'])
```

attr **JWT_REFRESH_CSRF_COOKIE_NAME** `class-attribute` `instance-attribute`

```
JWT_REFRESH_CSRF_COOKIE_NAME = 'csrf_refresh_token'
```

attr **JWT_REFRESH_CSRF_COOKIE_PATH** `class-attribute` `instance-attribute`

```
JWT_REFRESH_CSRF_COOKIE_PATH = '/'
```

attr **JWT_REFRESH_CSRF_FIELD_NAME** `class-attribute` `instance-attribute`

```
JWT_REFRESH_CSRF_FIELD_NAME = 'csrf_token'
```

attr **JWT_REFRESH_CSRF_HEADER_NAME** `class-attribute` `instance-attribute`

```
JWT_REFRESH_CSRF_HEADER_NAME = 'X-CSRF-TOKEN'
```

attr **JWT_QUERY_STRING_NAME** `class-attribute` `instance-attribute`

```
JWT_QUERY_STRING_NAME = 'token'
```

attr **JWT_JSON_KEY** `class-attribute` `instance-attribute`

```
JWT_JSON_KEY = 'access_token'
```

attr **JWT_REFRESH_JSON_KEY** `class-attribute` `instance-attribute`

```
JWT_REFRESH_JSON_KEY = 'refresh_token'
```

attr **JWT_IMPLICIT_REFRESH_ROUTE_EXCLUDE** `class-attribute` `instance-attribute`

```
JWT_IMPLICIT_REFRESH_ROUTE_EXCLUDE = Field(default_factory=list)
```

attr **JWT_IMPLICIT_REFRESH_ROUTE_INCLUDE** `class-attribute` `instance-attribute`

```
JWT_IMPLICIT_REFRESH_ROUTE_INCLUDE = Field(default_factory=list)
```

attr **JWT_IMPLICIT_REFRESH_METHOD_EXCLUDE** `class-attribute` `instance-attribute`

```
JWT_IMPLICIT_REFRESH_METHOD_EXCLUDE = Field(default_factory=list)
```

attr **JWT_IMPLICIT_REFRESH_METHOD_INCLUDE** `class-attribute` `instance-attribute`

```
JWT_IMPLICIT_REFRESH_METHOD_INCLUDE = Field(default_factory=list)
```

attr **JWT_IMPLICIT_REFRESH_DELTATIME** `class-attribute` `instance-attribute`

```
JWT_IMPLICIT_REFRESH_DELTATIME = timedelta(minutes=10)
```

attr **is_algo_symmetric** `property`

```
is_algo_symmetric
```

Check if the JWT_ALGORITHM is a symmetric encryption algorithm.

attr **is_algo_asymmetric** `property`

```
is_algo_asymmetric
```

Check if the JWT_ALGORITHM is an asymmetric encryption algorithm.

attr **private_key** `property`

```
private_key
```

Private key to encode token.

attr **public_key** `property`

```
public_key
```

Public key to decode token.

attr **previous_public_key** `property`

```
previous_public_key
```

Previous public key for key rotation fallback during decode.

Returns None when no previous key is configured.

attr model_config class-attribute

```
model_config = SettingsConfigDict(extra='forbid', arbitrary_types_allowed=True,
validate_default=True, case_sensitive=False, env_prefix='',
nested_model_default_partial_update=False, env_file=None, env_file_encoding=None,
env_ignore_empty=False, env_nested_delimiter=None, env_nested_max_split=None,
env_parse_none_str=None, env_parse_enums=None, cli_prog_name=None, cli_parse_args=None,
cli_parse_none_str=None, cli_hide_none_type=False, cli_avoid_json=False, cli_enforce_required=False,
cli_use_class_docs_for_groups=False, cli_exit_on_error=True, cli_prefix='', cli_flag_prefix_char='-
', cli_implicit_flags=False, cli_ignore_unknown_args=False, cli_kebab_case=False,
cli_shortcuts=None, json_file=None, json_file_encoding=None, yaml_file=None,
yaml_file_encoding=None, yaml_config_section=None, toml_file=None, secrets_dir=None,
protected_namespaces=('model_validate', 'model_dump', 'settings_customise_sources'),
enable_decoding=True)
```

meth has_location

```
has_location(location)
```

Check if the token location is enabled.

Source code in `authx/config.py`

```
120 def has_location(self, location: str) -> bool:
121     """Check if the token location is enabled."""
122     return location in self.JWT_TOKEN_LOCATION
```

meth settings_customise_sources classmethod

```
settings_customise_sources(settings_cls, init_settings, env_settings, dotenv_settings,
file_secret_settings)
```

Define the sources and their order for loading the settings values.

PARAMETER	DESCRIPTION
`settings_cls`	The Settings class. **TYPE:** `type[BaseSettings]`
`init_settings`	The `InitSettingsSource` instance. **TYPE:** `PydanticBaseSettingsSource`
`env_settings`	The `EnvSettingsSource` instance. **TYPE:** `PydanticBaseSettingsSource`
`dotenv_settings`	The `DotEnvSettingsSource` instance. **TYPE:** `PydanticBaseSettingsSource`
`file_secret_settings`	The `SecretsSettingsSource` instance. **TYPE:** `PydanticBaseSettingsSource`

RETURNS	DESCRIPTION
`tuple[PydanticBaseSettingsSource, ...]`	A tuple containing the sources and their order for loading the settings values.

Source code in `.venv/lib/python3.14/site-packages/pydantic_settings/main.py`

```python
@classmethod
def settings_customise_sources(
    cls,
    settings_cls: type[BaseSettings],
    init_settings: PydanticBaseSettingsSource,
    env_settings: PydanticBaseSettingsSource,
    dotenv_settings: PydanticBaseSettingsSource,
    file_secret_settings: PydanticBaseSettingsSource,
) -> tuple[PydanticBaseSettingsSource, ...]:
    """
    Define the sources and their order for loading the settings values.

    Args:
        settings_cls: The Settings class.
        init_settings: The `InitSettingsSource` instance.
        env_settings: The `EnvSettingsSource` instance.
        dotenv_settings: The `DotEnvSettingsSource` instance.
        file_secret_settings: The `SecretsSettingsSource` instance.

    Returns:
        A tuple containing the sources and their order for loading the settings values.
    """
    return init_settings, env_settings, dotenv_settings, file_secret_settings
```

March 20, 2026 March 19, 2026 GitHub